

In Labs 2-4 of ISSEM you will apply the concepts you have learned in the theoretical modules on a real world application, namely the Infant Incubator, described separately.

Lab 3

Risk Assessment:

In Lab 2 you wrote a preliminary threat model of the Infant Incubator model. In this Lab, you have to do a risk analysis of the threats that you found in the threat model. You may add new threats to the threat model but you must include risk assessment of all listed threats using the following Risk Table (example below follows from Lab 2). You can look at the lecture recordings for more information on Risk.

Risk Table

Severity

The vulnerability severity is determined using the likelihood and impact weights in the following table:

		Impact				
		<i>Minimal</i>	<i>Low</i>	<i>Medium</i>	<i>High</i>	<i>Critical</i>
Likelihood	<i>Critical</i>	Minimal	Low	Medium	High	Critical
	<i>High</i>	Minimal	Low	Medium	High	Critical
	<i>Medium</i>	Minimal	Low	Medium	Medium	High
	<i>Low</i>	Minimal	Low	Low	Low	Medium
	<i>Minimal</i>	Minimal	Minimal	Minimal	Low	Low

Student Portion

Risk Table

Threat No.	Brief Description	Likelihood	Impact	Risk	Do You Agree with the Risk Level Rating?
1	User Spoofing	Medium	High	Medium	Yes
2	...	...	...	...	...

Justifications

Threat Number 1 - User Spoofing (Example)

Likelihood is medium because the attacker has to be able to run the eavesdropping process on the client machine to snoop on the credentials.

Impact is high because the attacker can get access to the videos and then do unknown actions in terms of accessing the home, including breaking in.

Risk is rated by the table as Medium. We agree with the risk severity because it is not easy to run a spoofed process on a typical client, but the risk needs to be addressed (i.e. not low) because the danger from successful attack is quite severe.

Some Notes

- For each risk give a proper and credible justification for your threat analysis (likelihood, impact, risk level).
- Remember that in real life these threats will be read by a non-security expert and must be believable to them, so be as clear and comprehensive as possible. The example above is a very simple one but other threats may need better justification.
- Indicate any assumption you are making about the underlying system, e.g. 'Client machine has the usual security protections.'
- Feel free to use any references to support your conclusions but you cannot assume that the reader will read those references so your explanations must be self-evident even without the references.

Coding Task 1

You will find the steps to run the flask application and the Incubator temperature files in the README.md section of the zipped folder attached to the submission.

1. Find the SQL Injection of the login page. You need to submit a screenshot of the webpage after having logged in using the payload you find.
 2. Retrieve the plaintext password and submit the same.
 3. Please submit the code you use to decrypt the password and name it decrypt.py
 4. After the above tasks, you have to find at least 5 new vulnerabilities in the code.
- Please write up explanations identifying the vulnerability you find, and your approach towards fixing this vulnerability.

Notes

- SQL injection leaking the password does not count as one of your new vulnerabilities. Neither do the vulnerabilities from Coding Task 2.
- You should have a writeup PDF or jupyter notebook for Part 4 of this Coding Task. You may find a Jupyter notebook easier to work with when discussing code snippets with their vulnerabilities.
- Think about how you would test a fix to some of your vulnerabilities for part 4. It will help you in the next lab.
- One of your vulnerabilities must identify an issue between the interaction between `run()` and `processCommands()` and suggest a remedy.

Coding Task 2

We have listed hints for 3 vulnerabilities below which are currently present in code. In Lab 2 you explained how these vulnerabilities could be exploited. In this lab you are going to provide the patches to fix these vulnerabilities.

1. Password is Hardcoded

2. Token List has the potential to continuously grow.
3. Plaintext authentication token and authentication.

Notes

1. Just because the vulnerability originally appeared in one file does not mean that you only have to change that one file or one location to fix the vulnerability.
 - For instance, hardcoded passwords in `SampleNetworkClient.py` will also need changes in the `SampleNetworkServer.py`. So will plaintext tokens and authentication.
 - Full credit means fixing a vulnerability (e.g. hardcoding passwords) everywhere in the codebase.
2. You want to be able to test these fixes, so we have included copies of the old files in a `copy/` directory. The md5 SHAs for the original files are also included in a `SHA.txt` in that directory, in case you are unsure if you updated the old files accidentally. Make sure to make your changes outside of `copy/`.
3. Patching means patch notes. Include a description for each of your changes in a writeup in `writeup/`.
4. Remember that fixing a vulnerability means a systemic fix and enforcement. For instance, a change in one function in one file may require you to update other functions or the class variables. Document this in your patch notes.
 - For instance, for vulnerability 2, you must enforce that it is never large enough to crash the server by enforcing a size limit.

Submission

1. Provide the deliverables from the writeup portions of this lab in the `writeup/` directory in this directory. See the `writeup/INCLUDE.md` for a list of the writeup materials.
 - Do not have your writeups anywhere else in the file directory here but the `writeup/` folder.
3. Zip the entire directory, after including all writeups `writeup/` directory, and turn in the `.zip` file into Brightspace.
4. This is a group assignment. Submit your zip file with the title `lab3_group<group_number>.zip`. You may also include all files in your `writeup/` next to your `.zip` file, but make sure they are all in your `writeup/` folder. Good luck!